



SAPIENZA
UNIVERSITÀ DI ROMA

**FACOLTÀ DI INGEGNERIA
DELL'INFORMAZIONE, INFORMATICA E
STATISTICA**

MASTER DEGREE IN ELECTRONICS ENGINEERING

DISA II

**RTL Development and Verification of a Stand-Alone
32-bit Floating-Point Exponential Functional Unit**

Studenti:

Biagio Palmiero 1938749
Marco Masciolini 1951623
Emanuele Bassetta 1875681

ACADEMIC YEAR 2025-2026

Date: October 5, 2026

Contents

0.1	Dichiarazione sull'uso dell'Intelligenza Artificiale	2
1	Introduzione	4
1.1	L'Approccio Ibrido (Table-Driven Approach)	4
1.2	Metodo di Tang e Range Reduction	4
1.3	Approssimazione Polinomiale (Schema di Horner)	5
1.4	Gestione dei Casi Speciali IEEE-754	6
2	Interfaccia di Controllo (Livello Top-Level)	7
2.1	Entity e Segnali di Input/Output	7
2.2	Protocollo di Handshake (Valid/Ready)	7
3	Architettura Sequenziale Multiciclo	9
3.1	Diagramma ASM (Algorithmic State Machine)	9
3.2	Progetto del Datapath	11
3.3	Mappatura delle Eccezioni (Output Multiplexer)	12
4	Architettura Pipelined	13
4.1	Diagramma ASM (Algorithmic State Machine)	14
4.2	Progetto del Datapath	17
5	Verifica e Analisi dei Risultati (Architetture Sequenziale e Pipelined)	20
5.1	Simulazioni Temporali	21
5.2	Analisi degli Errori (ULP) rispetto alla Libreria Matematica .	25
6	Analisi delle Prestazioni e Trade-Off Architetturale	26
6.1	Discussione dei Risultati: Area vs Throughput	28

0.1 Dichiarazione sull'uso dell'Intelligenza Artificiale

Come richiesto dalla traccia del progetto, si documenta di seguito l'utilizzo di strumenti di Intelligenza Artificiale a supporto dello sviluppo:

1. **Nome e versione dell'IA:** Google Gemini 3.1 Pro High.

2. **Fasi del progetto in cui è stata usata:**

- Ricerca di progetti di riferimento come libreria `libm` e relativa spiegazione guidata del codice.
- Debug del prototipo iniziale del codice C.
- Spiegazione dell'implementazione di una LUT in VHDL.
- Ricerca di paper accademici per la comprensione della logica dell'**Hybrid (Table-Driven) Approach**, come ad esempio lo studio di Dalloo *et al.* [1].
- Ricerca dei coefficienti matematici e dei valori della LUT per l'algoritmo di Tang.
- Risoluzione di un problema di sincronizzazione (race condition) nel testbench.
- Comprensione del protocollo Handshake per la versione sequenziale. In particolare, mappatura dei segnali di ingresso ed uscita.
- Spiegazione su come ricavare i report di area e timing in ambiente Xilinx Vivado.
- Generazione del codice testuale Mermaid per importare la struttura base del diagramma ASM su draw.io.
- Correzione codice $\text{\LaTeX}$ per stesura della relazione.

3. **Prompt forniti all'IA (esempi principali):**

- *"Puoi spiegarmi il procedimento logico per estrarre i coefficienti di Horner e della LUT usati nella libreria `libm` di seguito allegata?"*
- *"Il testbench usa 'wait for 30 ns' per alzare `valid_i`, ma la FSM perde il segnale sul fronte di salita. Come risolvo?"*
- *"Riguardo il protocollo Handshake, perché in alcune documentazioni il segnale `ready_o` è raggruppato sotto le interfacce di input pur essendo fisicamente un output? Stesso dubbio per gli altri segnali di ingresso ed uscita."*
- *"Come si scrive una memoria ROM in linguaggio VHDL?"*
- *"Per il confronto relativo ad area e timing per entrambi le versioni mi conviene fare una stima qualitativa basandomi sui blocchi messi nel datapath oppure si deve fare tramite Vivado?"*

- *"Fammi uno schema del diagramma ASM dell'architettura sequenziale utilizzando il linguaggio mermaid in modo da importarlo su draw.io ed ottenere un grafico da poter mettere nella relazione."*
- *"Mi puoi spiegare come si definiscono e si utilizzano i tipi record in VHDL, e quali sono i principali vantaggi nell'utilizzarli per raggruppare segnali di controllo e dati correlati in un'interfaccia?"*
- *"Qual è la struttura generale di un modulo top-level in un sistema hardware sincrono a pipeline?"*
- *"Puoi spiegarmi nel dettaglio come funziona il protocollo di handshake basato sui segnali valid e ready in una pipeline, e come viene gestita la propagazione combinatoria della backpressure?"*
- *"Utilizzando questo tb, il segnale flags_ o sembra ignorare alcune eccezioni, dove può stare l'errore?"*
- *"In base alla logica del protocollo handshake, mi spieghi il miglior modo per modificare il tb in modo da verificare il corretto funzionamento della backpressure?"*
- *"Ho notato che nell'utilization report della versione pipelined manca lo stadio 5, come mai?"*

4. Modifiche apportate ai risultati dell'IA:

- I coefficienti matematici teorici sono stati quantizzati e convertiti manualmente dal team in formato a virgola fissa (Q8.24) sintetizzabile.
- I ritardi del testbench sono stati corretti e traslati manualmente (es. a 33 ns) per evitare il disallineamento dei delta-cycle.
- Lo schema Mermaid è stato importato su draw.io e poi riorganizzato, espanso e dettagliato manualmente dal team per creare il diagramma ASM della versione sequenziale.
- Correzione delle costanti C0, C1, C2 generate in modo allucinatorio.
- Modifica del testbench nella versione pipelined per correggere errori sui flag delle eccezioni.

1 Introduzione

L'obiettivo di questo progetto è il calcolo della funzione trascendente e^x , tramite l'implementazione hardware diretta di una Floating-Point Unit (FPU) esponenziale a 32-bit (singola precisione IEEE-754) in modalità *Stand-Alone*. La progettazione e l'implementazione RTL (Register Transfer Level) di questa unità sono state sviluppate valutando attentamente i compromessi (trade-off) in termini di area e latenza per ciascuna delle due versioni implementate: una sequenziale multiciclo e una con pipeline a pieno throughput.

1.1 L'Approccio Ibrido (Table-Driven Approach)

Per il nostro hardware è stato scelto un *Approccio Ibrido (Table-Driven)*, in modo da ottenere un bilanciamento ottimale tra utilizzo delle risorse hardware e latenza computazionale [1]. L'approssimazione polinomiale di una funzione esponenziale, nello specifico la serie di Taylor, ha come drawback una convergenza troppo lenta per esponenti troppo grandi. Per questo motivo, deve essere accompagnata da una *Range Reduction* iniziale dell'input. Il flusso logico implementato, dunque, si articola in quattro fasi fondamentali:

- Riduzione dell'input a un intervallo ristretto, confinando il dominio di calcolo in uno spazio aritmetico limitato.
- Utilizzo di una LUT (Look-Up Table) per ottenere una stima iniziale del valore esponenziale.
- Utilizzo di un'approssimazione polinomiale (basata sul metodo di Tang) valutata tramite lo *Schema di Horner* per la computazione dei bit meno significativi dell'esponenziale.
- Ricostruzione del risultato finale completo, stimato in modo meno dispendioso in termini di hardware rispetto all'applicazione individuale di solo uno dei due metodi (*LUT e Approssimazione Polinomiale*).

1.2 Metodo di Tang e Range Reduction

In un'architettura hardware, calcolare direttamente la funzione esponenziale consumerebbe troppe risorse hardware. L'algoritmo di Tang risolve questo problema riconducendo l'esponenziale in base e a un'esponenziale in base 2. Sfruttando le proprietà dei logaritmi, l'argomento di ingresso x viene inizialmente moltiplicato per la costante fissa $\log_2(e)$. Abbiamo:

$$e^x = 2^{x \cdot \log_2(e)} = 2^y \quad (1)$$

A questo punto si applica la vera e propria *Range Reduction*. Il nuovo valore y viene scomposto matematicamente nella sua parte intera e nella sua componente frazionaria:

$$y = N + f \quad (2)$$

dove N è un intero e $f \in [0, 1)$ è la frazione. Per facilitare la successiva approssimazione polinomiale, la porzione frazionaria f subisce un'ulteriore riduzione. Essa viene frazionata in un termine discreto a passi fissi e in un resto molto piccolo:

$$f = \frac{j}{32} + r \quad (3)$$

dove $j \in [0, 31]$ è un intero che rappresenta i 5 bit più significativi della frazione, e $r \in [0, 1/32)$ rappresenta il residuo frazionario.

L'equazione esponenziale originale viene quindi riformulata e trasformata nel prodotto di tre termini matematici distinti e molto più semplici da calcolare:

$$2^y = 2^N \cdot 2^{j/32} \cdot 2^r \quad (4)$$

Ognuno di questi termini verrà risolto con un approccio hardware dedicato:

- 2^N rappresenta la parte intera e non richiede calcoli complessi: il numero intero N viene inviato ad un addizionatore e sommato al *bias* di 127 in modo da ottenere il nuovo campo esponente del risultato IEEE-754.
- $2^{j/32}$ rappresenta la stima iniziale del valore esponenziale che possiede solo 32 combinazioni possibili e può essere pre-calcolato in una Look-Up Table.
- 2^r è la più piccola parte frazionaria dell'esponenziale, calcolata rapidamente tramite un'approssimazione polinomiale secondo lo schema di *Horner*.

1.3 Approssimazione Polinomiale (Schema di Horner)

La valutazione dell'ultimo termine frazionario 2^r necessita di un'approssimazione su un dominio molto piccolo.

Per garantire la massima precisione senza dover calcolare l'intera serie di Taylor (che richiederebbe troppi termini e quindi troppe risorse hardware), il nostro progetto si basa sull'utilizzo di coefficienti polinomiali pre-calcolati. In particolare, sono state estratte le costanti C_0, C_1, C_2 dalla libreria matematica standard `libm` (ambiente C).

Definiti i coefficienti, l'approssimazione assume la forma di un polinomio di secondo grado:

$$P(r) \approx 2^r = C_2 \cdot r^2 + C_1 \cdot r + C_0 \quad (5)$$

Per ottimizzare il calcolo di questa equazione in hardware, minimizzando il numero di moltiplicatori fisici necessari, il polinomio è stato fattorizzato secondo lo schema di Horner, raccogliendo la variabile indipendente r :

$$P(r) = (C_2 \cdot r + C_1) \cdot r + C_0 \quad (6)$$

Come si evince dall'equazione, la riscrittura secondo Horner permette di calcolare l'intero polinomio eseguendo unicamente 2 moltiplicazioni e 2 addizioni, eliminando del tutto la necessità di calcolare esplicitamente le potenze della variabile r .

1.4 Gestione dei Casi Speciali IEEE-754

Ciascuna eccezione è stata mappata su uno specifico bit del vettore *flags_o*, in particolare, al verificarsi di un determinato caso speciale il bit ad esso associato assumerà il valore logico '1' altrimenti '0'.

I casi limite della nostra unità hardware sono:

- **0xUF (Underflow)**: mappato sul bit 1. Viene impostato a '1' quando il risultato è troppo piccolo per essere rappresentato.
- **0xOF (Overflow)**: mappato sul bit 2. Viene impostato a '1' quando il risultato eccede il limite massimo rappresentabile dai 32-bit in formato IEEE-754 a singola precisione (es. input $x > 88.72$).
- **0xNV (Invalid Operation)**: mappato sul bit 4. Viene impostato a '1' se in ingresso viene fornito un operando non valido, come un *Not-a-Number* (NaN).
- **0xDZ (Divide by Zero)**: mappato sul bit 3. Incluso per garantire la piena conformità all'interfaccia standard IEEE-754. Poiché l'architettura non prevede divisioni, questo bit è cablato a massa ('0').
- **0xNX (Inexact)**: mappato sul bit 0. Viene impostato a '1' per indicare che la funzione trascendente ha subito un arrotondamento o troncamento rispetto al valore reale. Lo standard richiede che sia alto sia quando si verificano i casi speciali overflow e/o underflow, sia quando in ingresso è presente un float generico.

2 Interfaccia di Controllo (Livello Top-Level)

2.1 Entity e Segnali di Input/Output

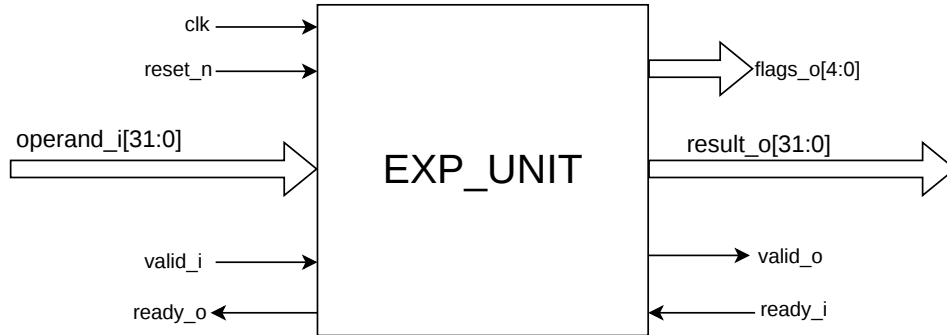


Figure 1: Interfaccia (Top-level Entity) della FPU Esponenziale.

In Figura 1 è riportato il nostro blocco hardware incapsulato all'interno di una singola *entity*. Il nostro IP Core *Stand-Alone* è progettato per funzionare come unità plug-and-play a sé stante. Riceve in ingresso un operando a 32-bit (formato IEEE-754 a singola precisione) tramite il bus **operand_i** e fornisce in uscita il risultato a 32-bit, sempre nello stesso formato, tramite il bus **result_o**. È stato inoltre inserito un bus di uscita a 5-bit, **flags_o**, per la segnalazione delle eccezioni attive. L'architettura è ovviamente provvista di **clk** e di un segnale di **reset_n** attivo basso per l'inizializzazione asincrona. Infine, per l'implementazione del protocollo Handshake, come segnali di ingresso sono stati impostati **valid_i** e **ready_i**, mentre come segnali di uscita **valid_o** e **ready_o**.

2.2 Protocollo di Handshake (Valid/Ready)

I quattro segnali di controllo sono stati implementati per rispettare il protocollo di comunicazione AXI-Stream. In particolare:

- **valid_i**: Il dispositivo a monte (*master*) presenta un operando valido in ingresso all'unità hardware.
- **ready_i**: Il dispositivo a valle (*slave*) è pronto, in questo ciclo di clock, ad acquisire il dato in uscita dall'unità hardware.
- **valid_o**: L'unità hardware fornisce il risultato in questo ciclo di clock.
- **ready_o**: L'unità hardware è pronta ad accettare un nuovo operando in questo ciclo di clock.

L'interfaccia si basa su un meccanismo di *Handshake* bidirezionale *Master-Slave*, dove il trasferimento dei dati avviene esclusivamente sul fronte di salita del clock in cui entrambi i segnali **valid** e **ready** sono alti.

3 Architettura Sequenziale Multiciclo

3.1 Diagramma ASM (Algorithmic State Machine)

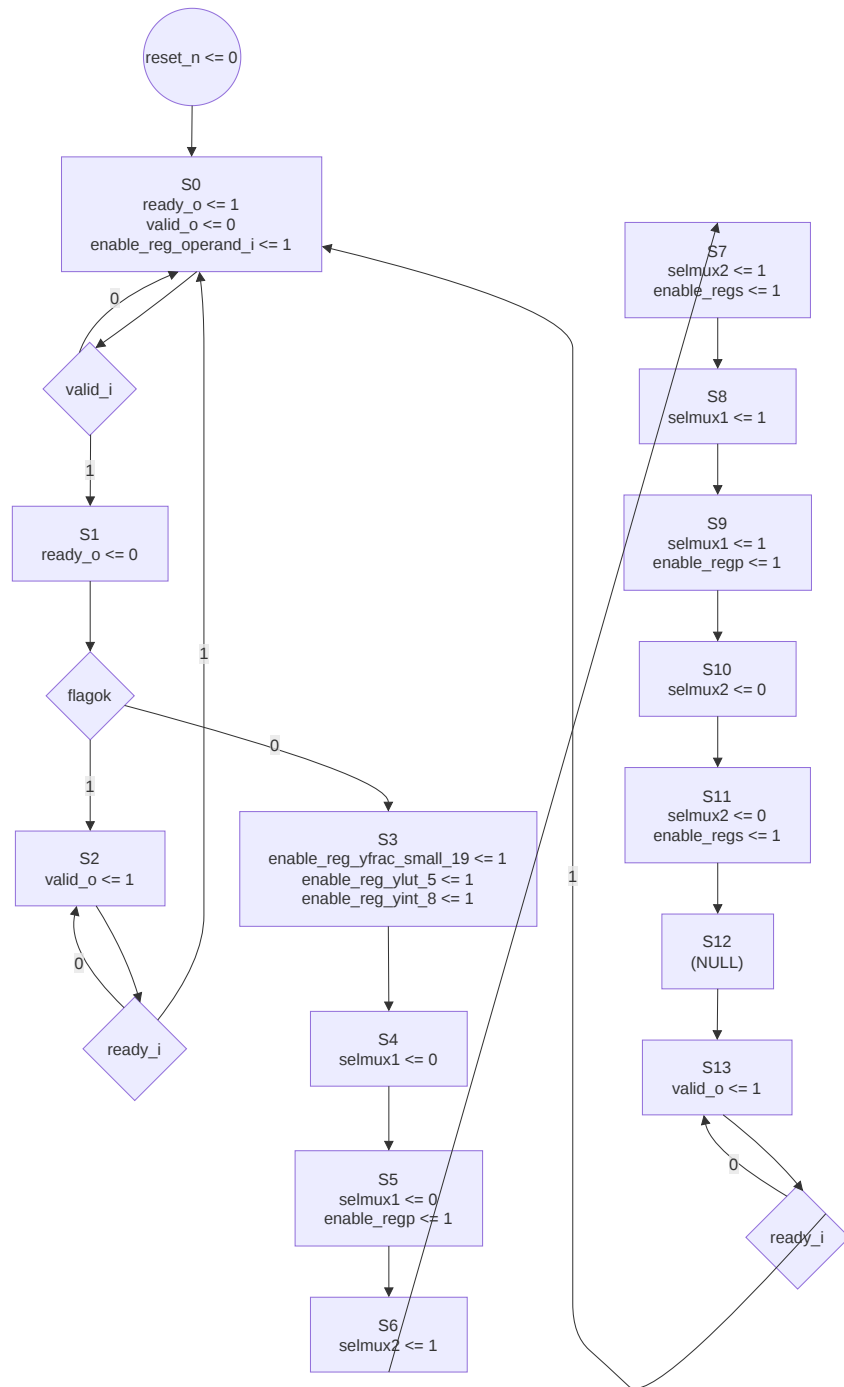


Figure 2: Diagramma ASM del flusso di controllo logico.

In figura 2 è mostrato il diagramma ASM del flusso di controllo logico da noi progettato. A partire dallo stato iniziale **S0**, poiché l'unità hardware è pronta a ricevere un nuovo dato in ingresso in questo ciclo di clock (**ready_o** = '1'), la nostra FSM procede ad abilitare il registro **reg_operand_i** in modo tale da fargli acquisire il dato in ingresso **operand_i** a 32 bit. Poiché il dispositivo a monte ha presentato un dato valido in ingresso all'unità hardware, la FPU inizia a lavorare e non è più disponibile a ricevere un nuovo dato in questo ciclo di clock (**ready_o** = '0').

Se l'operando in ingresso viene riconosciuto come eccezione, il segnale **flagok** si alza, viene presentato il risultato in uscita (**valid_o** = '1') e, qualora fosse acquisito dal dispositivo a valle (**ready_i** = '1'), il percorso termina e si torna nello stato iniziale **S0**. Nel caso in cui invece **flagok** fosse basso, si procede, tramite l'applicazione dell'algoritmo controllato dalla nostra FSM, al calcolo del risultato in formato IEEE-754 a singola precisione per un generico float.

Anche in questo caso, i vincoli imposti dal protocollo Handshake sono rispettati. Vediamo come, in presenza di un'eccezione, l'unità hardware produca un risultato valido dopo soli 3 cicli di clock, mentre per un float generico dopo 13 cicli di clock (lo stato **S2** viene saltato). Dunque, avendo impostato una frequenza di clock di 100 MHz, il risultato nel primo caso sarà disponibile dopo 30 ns, mentre nel secondo caso dopo 130 ns (0.13 micro-secondi).

3.2 Progetto del Datapath

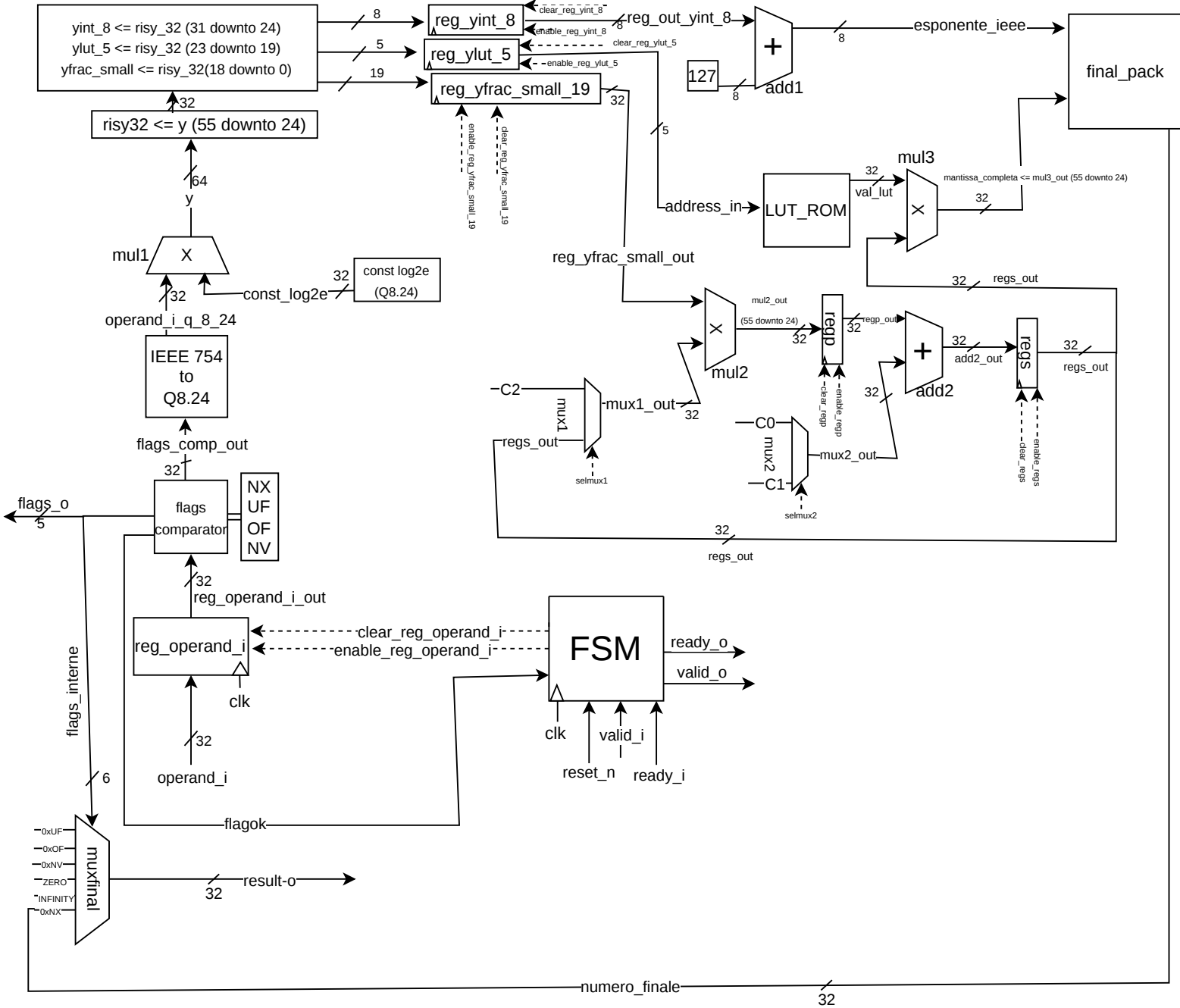


Figure 3: Schema architetturale dettagliato del Datapath Multiciclo.

In Figura 3 è possibile visualizzare il datapath completo dell'unità sequenziale. Lo schema può essere diviso in tre macro-aree:

- **Verifica & Unpack:** In questa sezione il numero in ingresso in formato virgola mobile a singola precisione IEEE-754 viene confrontato tramite il `flags_comparator` per verificare se è necessario mandare o meno un'eccezione in uscita. Qualora l'input non fosse riconosciuto come eccezione, esso viene inviato a un blocco combinatorio capace di convertirlo da formato IEEE-754 a formato a virgola fissa Q8.24, necessario per effettuare calcoli algebrici tra numeri binari.
- **Algoritmo:** Questa fase riguarda la parte meno significativa del prodotto tra `operand_i_q_8_24` e `const_log2e`, costituita da 19 bit rappresentanti la più piccola parte frazionaria di y . Come esplicitato in precedenza, il valore di 2^r viene calcolato tramite l'approssimazione polinomiale di Taylor secondo lo schema di Horner. Vediamo dunque come questi 19 bit (`reg_yfrac_small_out`) entrino in un macro-blocco combinatorio costituito da due multiplexer (`mux1`, `mux2`), un adder (`add2`) e un moltiplicatore (`mul2`). L'applicazione dello schema di Horner permette di calcolare l'intero polinomio eseguendo solo 2 moltiplicazioni e 2 addizioni che, grazie all'applicazione del resource sharing, vengono effettuate da un solo adder e un solo moltiplicatore aventi in ingresso le uscite di due multiplexer opportunamente pilotati dalla nostra FSM.
- **Packaging Finale:** In questa fase finale il numero viene ricostruito in formato IEEE-754. Essendo l'esponentiale sempre positivo viene messo uno '0' alla posizione 31 del `numero_finale`; l'esponente sommato al bias ed estrapolato in precedenza, insieme alla mantissa finale, vengono riposti nelle opportune posizioni.

3.3 Mappatura delle Eccezioni (Output Multiplexer)

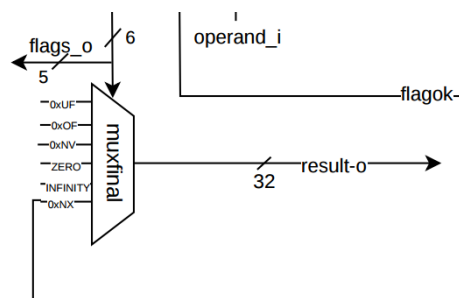


Figure 4: Dettaglio del multiplexer di uscita (`muxfinal`).

Come mostrato in Figura 4, i risultati delle eccezioni vengono selezionati in uscita da un multiplexer pilotato da un segnale di selezione chiamato

flags_interne. Tale segnale è un vettore a 6 bit, di cui i bit da 0 a 4 costituiscono il vettore in uscita **flags_o** per l'identificazione standard delle eccezioni. Si è scelto dunque di avere un selettore interno a 6 bit invece di 5, in modo tale da poter inserire due casi notevoli aggiuntivi: **INFINITY** e **ZERO**. Grazie a questa scelta progettuale, infatti, qualora il numero in ingresso sia zero o infinito, esso non viene processato dall'algoritmo del datapath, ma si restituisce direttamente in uscita il risultato esatto ad esso associato. Il mapping completo delle eccezioni è riportato in Tabella 1:

Identificativo (Eccezione)	flags_interne (6 bit)	flags_o (5 bit)	Output (result_o)
0xUF (Underflow)	"000011"	"00011" (0x03)	x"00000000" (Zero)
0xOF (Overflow)	"000101"	"00101" (0x05)	x"7F800000" (+Infinito)
0xNV (NaN)	"010000"	"10000" (0x10)	x"7FC00000" (NaN)
ZERO (± 0.0)	"100000"	"00000" (0x00)	x"3F800000" (+1.0)
INFINITY ($+\infty$)	"000000"	"00000" (0x00)	x"7F800000" (+Infinito)
0xNX (Normale)	"000001"	"00001" (0x01)	numero_finale

Table 1: Mapping delle eccezioni sul multiplexer finale.

4 Architettura Pipelined

L'obiettivo principale della versione pipelined è l'ottimizzazione delle prestazioni temporali del circuito per il calcolo della funzione esponenziale, conservando la stessa logica di algoritmo.

A differenza dell'approccio sequenziale, in cui l'intera mole di calcolo si sviluppa in un unico e lungo percorso critico (critical path), questa architettura suddivide il carico computazionale in stadi separati, i quali presentano ognuno un hardware dedicato ed inserisce dei registri tra di essi. Questa scelta progettuale riduce drasticamente la lunghezza del critical path, permettendo di abbassare il periodo minimo di clock (Tck) e, di conseguenza, innalzare la frequenza operativa massima del sistema. Come naturale compromesso architetturale, l'inserimento dei registri comporta un aumento della latenza di calcolo che si assesta a 5 cicli di clock per ottenere il primo risultato. Questa penalità apparente è ampiamente compensata da un netto incremento del throughput, il quale, a regime, garantisce l'elaborazione di un dato per ogni colpo di clock. Infine, per gestire in modo sicuro ed affidabile l'avanzamento dei dati attraverso gli stadi, evitando perdite di informazione o sovrascritture, l'architettura è governata da un protocollo di handshake valid-ready: questo meccanismo permette all'intero sistema di fermarsi in caso di stallo di una qualche operazione, attuando la backpressure, ossia propagando lo stallo all'indietro, stadio dopo stadio. Nello specifico, il protocollo si basa sull'interscambio asincrono di segnali di valid e ready tra stadi adiacenti. Ogni stadio della pipeline presenta 4 segnali di controllo:

- input_valid: il blocco a monte presenta dei dati validi.

- `input_ready`: lo stadio accetta nuovi dati nel ciclo corrente.
- `output_valid`: lo stadio presenta dei dati validi al blocco a valle.
- `output_ready`: il blocco a valle accetta nuovi dati nel ciclo corrente.

Uno stadio accetta i dati quando è vuoto, quindi non ci sono dati validi (`output_valid='0'`), oppure quando è pieno e vengono prelevati i dati svuotandolo nello stesso ciclo (`output_ready='1'`). Dunque si ha:

$$input_ready = !output_valid \quad || \quad output_ready$$

Da sottolineare come lo stadio non alterni carica e scarica dei dati, infatti viene svuotato mentre accetta nuovi dati, evitando di creare bolle. Successivamente, quando sono alti sia `input_valid` che `input_ready`, il nuovo dato viene campionato per essere processato (`register_en='1'`). Al ciclo di clock seguente si alza `output_valid`. In figura 5 lo schema logico del protocollo handshake.

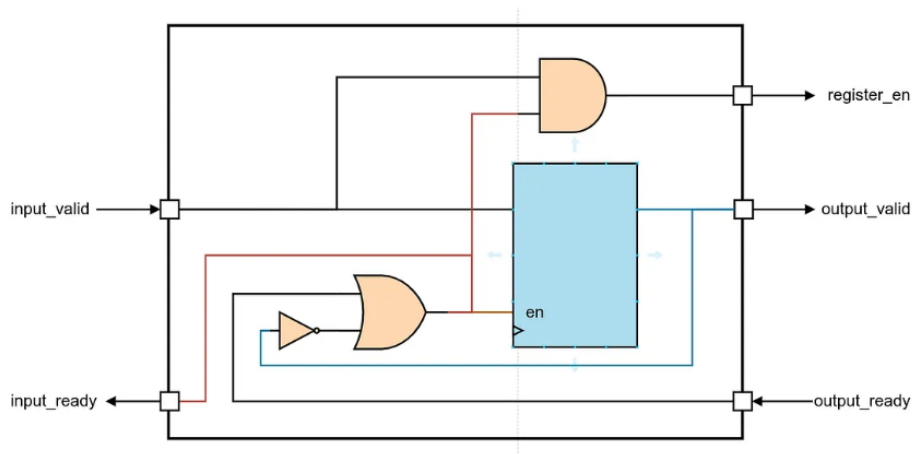


Figure 5: Schema logico del protocollo handshake.

4.1 Diagramma ASM (Algorithmic State Machine)

Il comportamento di controllo di ciascun blocco della pipeline è gestito da un'unica topologia di Finite State Machine (FSM), strutturata su due soli stati e replicata per tutti i 5 stadi. Come illustrato nel diagramma ASM, la transizione tra gli stati è governata unicamente dai segnali del protocollo valid-ready, mentre la logica combinatoria matematica, che varia per ogni stadio, viene generalizzata dalla funzione "calcolo_combinatorio" (figura 6).

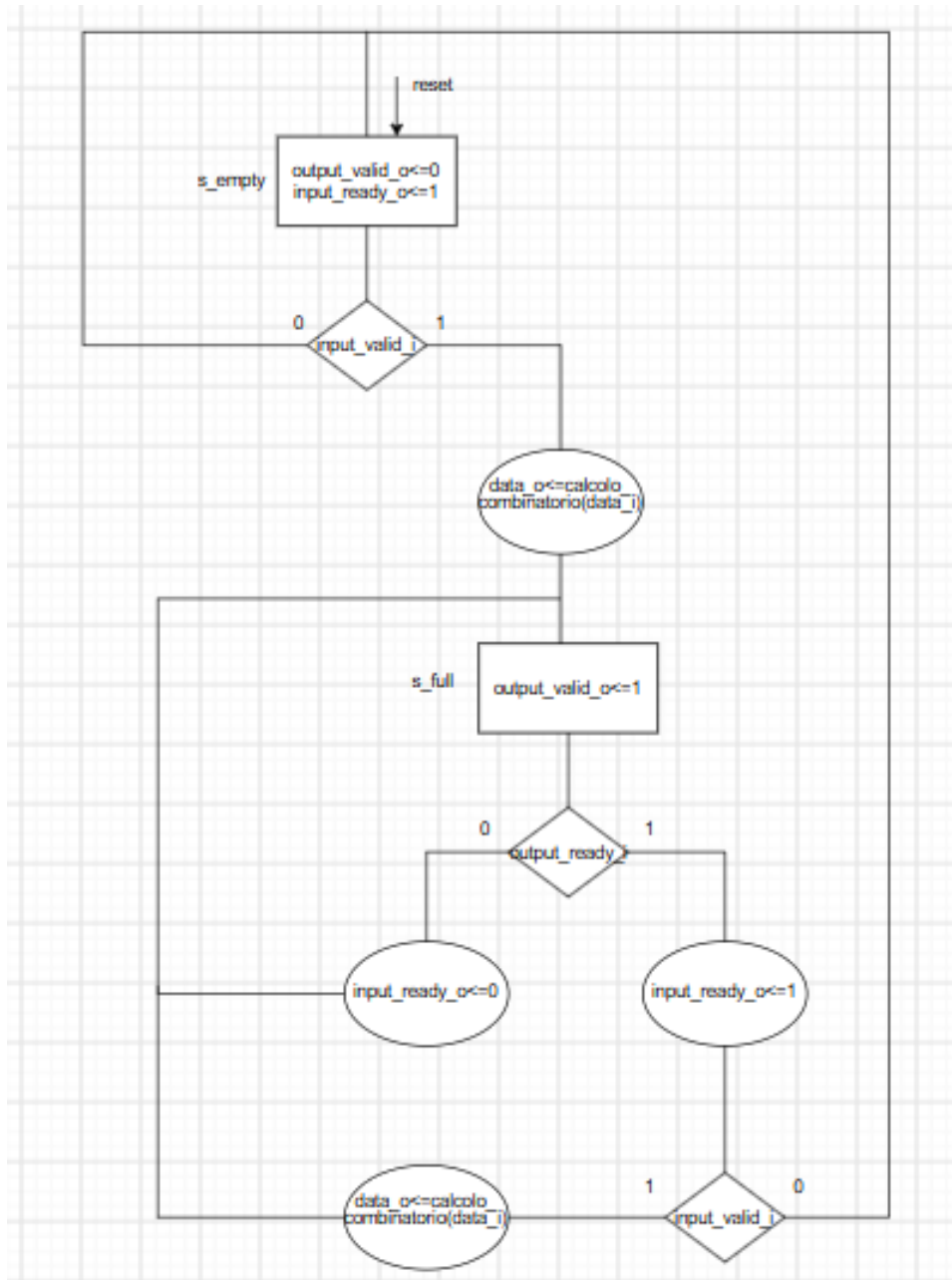


Figure 6: Diagramma ASM relativo al singolo stadio della struttura pipelined.

Il primo stato è "s_empty", in cui lo stadio non contiene dati validi ($\text{output_valid_o} = '0'$) ed è pronto ad accettarne di nuovi, comunicando a monte la sua disponibilità ($\text{input_ready_o} = '1'$). La FSM permane in questo stato in caso di reset basso, o finché non si presenta un dato valido in ingresso ($\text{input_valid_i} = '1'$). In tale istante, il blocco combinatorio esegue

il calcolo, il risultato viene campionato dai registri (`register_en = '1'`) e la macchina transita in `"s_full"`.

Nel secondo stato `"s_full"`, lo stadio contiene un risultato pronto per il blocco successivo (`output_valid_o = '1'`). L'evoluzione da questo stato dipende dalla condizione di `ready` del blocco a valle (`output_ready_i`):

- se a valle non vi è disponibilità (`output_ready_i = '0'`), lo stadio entra in backpressure. Abbassa immediatamente il proprio segnale `input_ready_o` per bloccare i dati a monte e cicla su se stesso, mantenendo intatto il dato memorizzato.
- se il blocco a valle è pronto a leggere (`output_ready_i = '1'`), lo stadio segnala a monte la capacità di ricevere un nuovo dato (`input_ready_o = '1'`). A questo punto, se è presente un nuovo ingresso valido, lo campiona e rimane in `"s_full"` (garantendo il throughput continuo). Se invece non ci sono nuovi dati in arrivo (`input_valid_i = '0'`), lo stadio si svuota e ritorna in `"s_empty"`.

Nello specifico, la rete combinatoria dedicata `"calcolo_combinatorio"` per ogni stadio è costituita da:

- Stadio 1: conversione dell'ingresso da IEEE 754 a Q8.24, calcolo nuovo esponente $y = x \cdot \log_2(e)$ per il cambio di base $e^x = 2^y$, mappatura dei flag delle eccezioni per operandi in ingresso 0, +inf, -inf, NaN.
- Stadio 2: estrazione della parte intera *yint*, dei 5 bit più significativi della parte frazionaria *y_lut* e della parte frazionaria residua *yfrac_small_19*, estrazione *val_lut* dalla ROM, riallineamento a 32 bit di *yfrac_small_19* (*yfrac_32*) e calcolo di $p_a = yfrac_{32} \cdot c2$.
- Stadio 3: calcolo $p_b = p_a_{32} + c1$, $p_c = yfrac_{32} \cdot p_b$.
- Stadio 4: calcolo $p_d = p_c_{32} + c0$, $m_{64} = val_{lut} \cdot p_d$, estrazione mantissa_32.
- Stadio 5: calcolo $esponente_{ieee} = yint + const_{127}$, assemblaggio numero_finale con mantissa_24, esponente_ieee e bit di segno, mux finale per selezionare eventuali eccezioni.

4.2 Progetto del Datapath

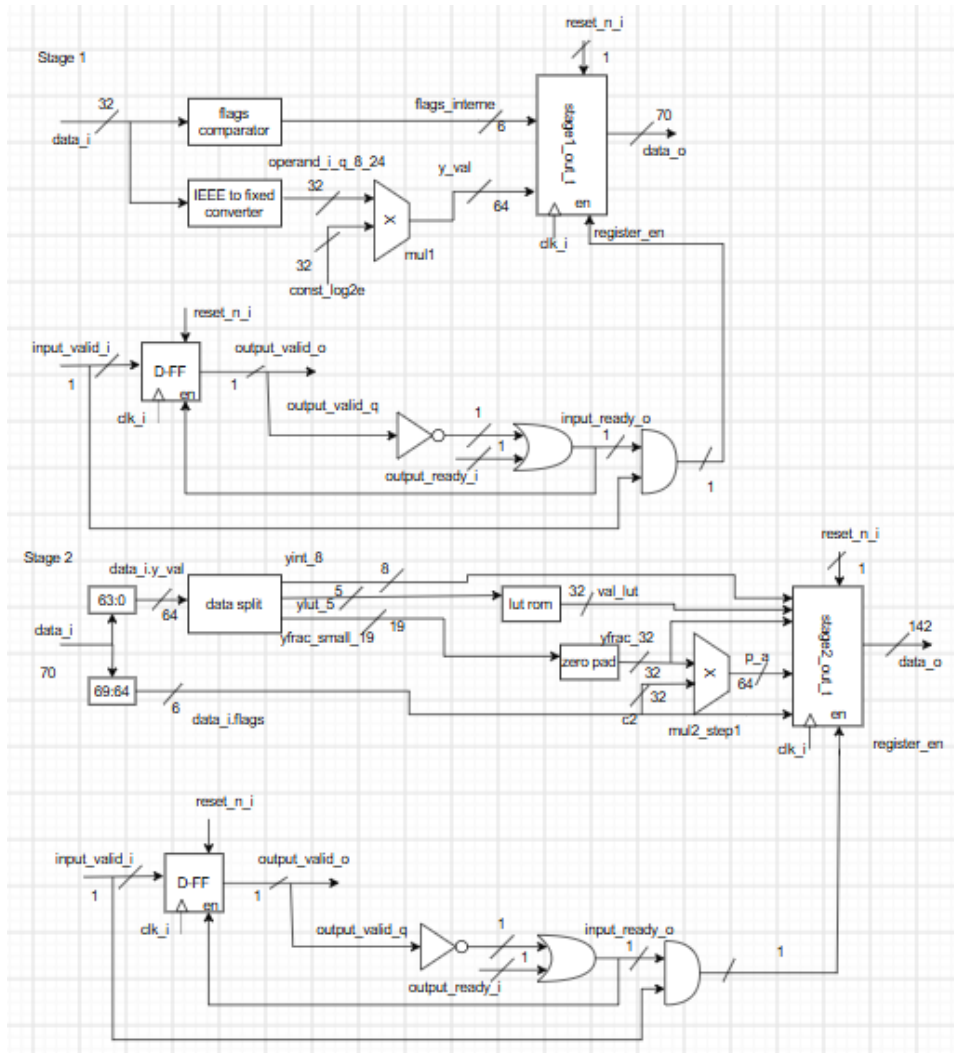


Figure 7: Datapath degli stadi 1 e 2.

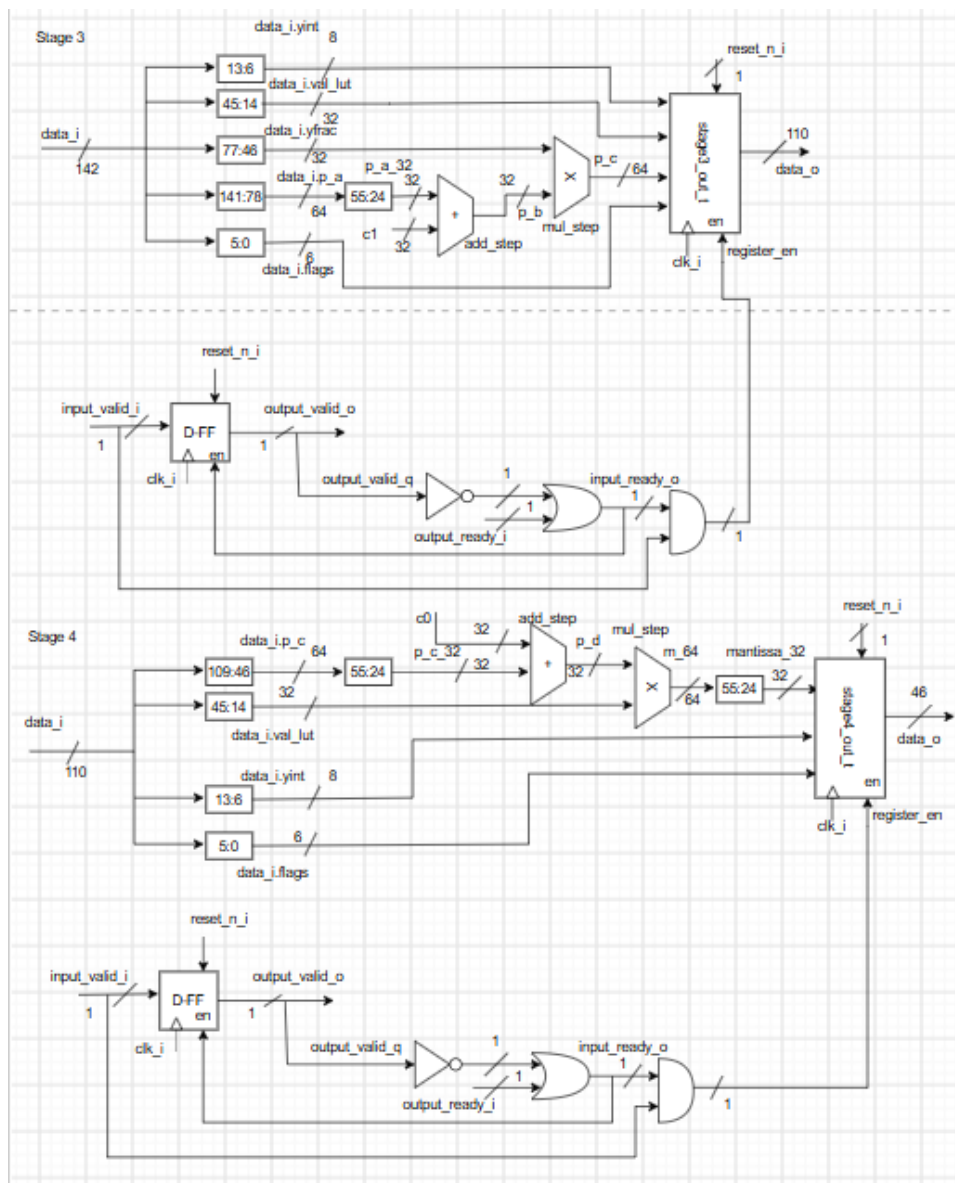


Figure 8: Datapath degli stadi 3 e 4.

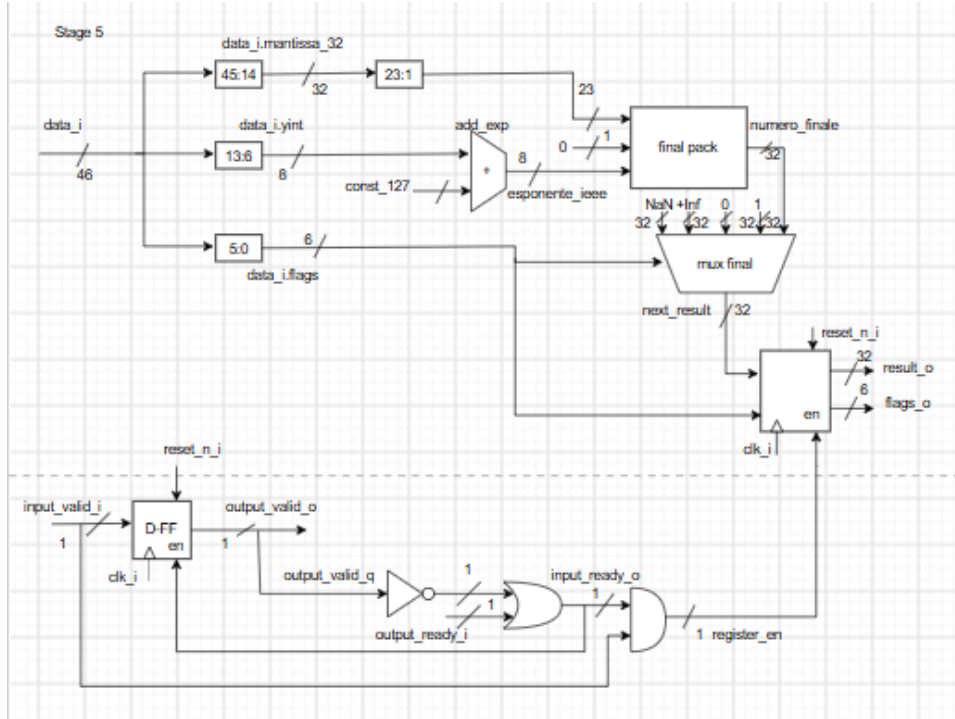


Figure 9: Datapath dello stadio 5.

L'architettura completa è stata assemblata all'interno di un modulo top-level puramente strutturale. Questo file ha il compito di istanziare in cascata i cinque stadi della pipeline, distribuire i segnali globali di clock e reset in parallelo a tutti i banchi di registri e instradare correttamente le interconnessioni.

Per ottimizzare ulteriormente la gestione del datapath e garantire un codice RTL chiaro ed ordinato, i bus di comunicazione tra gli stadi sono stati modellati sfruttando il costrutto VHDL "record", che svolge la funzione di un bus aggregato personalizzato, raggruppando i dati e i flag nel passaggio da uno stadio all'altro. In questo modo, si semplifica drasticamente il port mapping nel top-level, riducendo il rischio di errori di cablaggio ed incongruenze nel parallelismo dei bus.

Come si nota, ogni stadio presenta la separazione strutturale tra il datapath specifico e la logica di controllo del protocollo handshake, la quale resta sempre invariata. I segnali del protocollo, infatti, sono volutamente mantenuti esterni al record e dichiarati come porte indipendenti, la cui mappatura strutturale nel top-level chiude l'anello della backpressure: `output_valid` dello stadio N pilota direttamente l'`input_valid` dello stadio $N + 1$, mentre l'`input_ready` dello stadio $N + 1$ retroaziona sull'`output_ready` dello stadio N .

Un elemento fondamentale dell'architettura è la presenza dei registri di separazione (pipeline registers) posti al confine tra i vari stadi. Questi banchi di flip-flop hanno il compito di memorizzare fisicamente l'intero record elaborato dal datapath combinatorio. L'aggiornamento di tali registri è rigorosamente governato dal segnale di abilitazione `register_en`, generato localmente dalla logica di controllo di ciascuno stadio. Il campionamento del nuovo dato non avviene a ogni colpo di clock, ma esclusivamente quando il protocollo di handshake lo consente: questo meccanismo di clock gating logico sui registri è essenziale per congelare lo stato della pipeline durante una condizione di backpressure, impedendo la perdita o la sovrascrittura accidentale dei risultati in attesa di essere propagati.

Complessivamente, l'interconnessione strutturale dei cinque stadi determina una latenza architetturale fissa di 5 cicli di clock tra il campionamento dell'operando iniziale e l'emissione del corrispondente risultato valido in standard IEEE 754.

5 Verifica e Analisi dei Risultati (Architetture Sequenziale e Pipelined)

Poiché l'architettura Pipelined condivide con la versione Sequenziale il medesimo datapath matematico (costanti operative `C0`, `C1`, `C2`, Lookup Table e polinomio di Horner), la validazione numerica dei risultati presentata in questa sezione è applicabile e formalmente valida per entrambe le implementazioni. L'inserimento dei registri di stage nella pipeline modifica il throughput (portandolo a 1 operando/ciclo) e introduce una latenza fissa, ma mantiene una totale invarianza dei bit del dato calcolato.

5.1 Simulazioni Temporali

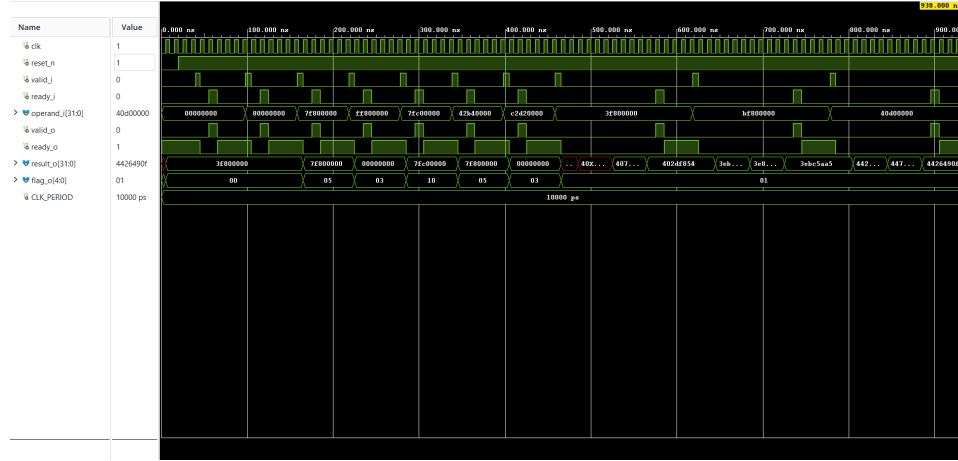


Figure 10: Forma d'onda iniziale che mostra il corretto superamento dei primi dieci vettori di test.

In Figura 10 è mostrata la forma d'onda ottenuta simulando la nostra FPU in ambiente Xilinx Vivado. Al fine di dimostrare la robustezza dell'architettura, il set di test iniziale è stato strutturato in dieci vettori: i primi sette sono stati scelti appositamente per sollecitare la logica di gestione delle eccezioni (*Corner Cases*), mentre gli ultimi tre rappresentano calcoli su float generici che attraversano l'intero datapath matematico. Nella Tabella 2 si riportano dettagliatamente i casi analizzati, confrontando l'output matematico atteso (generato dal Golden Model C di riferimento) con l'output hardware effettivamente ottenuto dalla simulazione RTL:

N.	Input (Esadecimale / Decimale)	Categoria	Output Atteso (Modello C)	Output Ottenuto (VHDL)	Flag Attive (flags_o)
1	0x00000000 (+0.0)	Zero	0x3F800000	0x3F800000	0x00 (Nessuna)
2	0x80000000 (-0.0)	Zero	0x3F800000	0x3F800000	0x00 (Nessuna)
3	0x7F800000 (+∞)	Infinito	0x7F800000	0x7F800000	0x05 (Overflow + Inexact)
4	0xFF800000 (-∞)	Infinito	0x00000000	0x00000000	0x03 (Underflow + Inexact)
5	0x7FC00000 (NaN)	Not-a-Number	0x7FC00000	0x7FC00000	0x10 (Invalid Operation)
6	0x42B40000 (+90.0)	Overflow (> 88.72)	0x7F800000	0x7F800000	0x05 (Overflow + Inexact)
7	0xC2D20000 (-105.0)	Underflow (< -104)	0x00000000	0x00000000	0x03 (Underflow + Inexact)
8	0x3F800000 (+1.0)	Float Generico	0x402DF854	0x402DF854	0x01 (Inexact)
9	0xBF800000 (-1.0)	Float Generico	0x3EBC5AA5	0x3EBC5AA5	0x01 (Inexact)
10	0x40D00000 (+6.5)	Float Generico	0x4426490F	0x4426490F	0x01 (Inexact)

Table 2: Confronto vettori di test: Risultati teorici software vs Risultati simulati hardware.

Come si evince dalla Tabella 2, l'hardware gestisce alla perfezione tutte le eccezioni (*corner cases*), fornendo immediatamente il risultato esatto codificato dallo standard IEEE-754 e bypassando il datapath logico. Nei casi di input su float generici (Vettori 8, 9 e 10), si osserva come l'output prodotto dalla simulazione hardware coincida esattamente al singolo bit con l'output

generato dal Golden Model C. Il sistema alza coerentemente la flag di *Inexact*, segnalando che il risultato fixed-point è affetto da una fisiologica perdita di precisione (intrinseca alla conversione in formato Q8.24) rispetto a un ipotetico calcolo fluttuante a precisione infinita.

Per validare ulteriormente la stabilità e l'accuratezza *Bit-Accurate* del datapath, è stato condotto un secondo test di simulazione. La FPU è stata sollecitata con dieci input floating-point generati casualmente, estratti da un bacino software di novanta test vectors di validazione.

N.	Input Hex	Output Atteso (Modello C)	Output Ottenuto (VHDL)
1	0xC188F981	0x331D6A13	0x331D6A13
2	0xC13EA097	0x36E0A241	0x36E0A241
3	0x407D4D00	0x42515F8B	0x42515F8B
4	0x419EB83C	0x4DC52165	0x4DC52165
5	0x40C61F68	0x43F43F93	0x43F43F93
6	0xC18DD1F	0x32AADE0D	0x32AADE0D
7	0xC16080B8	0x35584E6D	0x35584E6D
8	0x3D6CE800	0x3F879F1D	0x3F879F1D
9	0x40FA859C	0x451CFEFD	0x451CFEFD
10	0x4110558C	0x46014864	0x46014864

Table 3: Simulazione vettori randomici: la totale assenza di discrepanze conferma l'accuratezza del progetto RTL.

I risultati, riassunti nella Tabella 3, confermano in modo inequivocabile la solidità del progetto. Per tutti i campioni casuali testati, l'hardware e il modello software restituiscono output identici. Questa totale assenza di discrepanze certifica la bontà della sintesi RTL, dimostrando che l'architettura implementata replica in modo matematicamente impeccabile l'algoritmo di riferimento.

Di seguito viene riportata la forma d'onda ottenuta simulando la versione pipelined della FPU in ambiente Xilinx Vivado (figura 11).

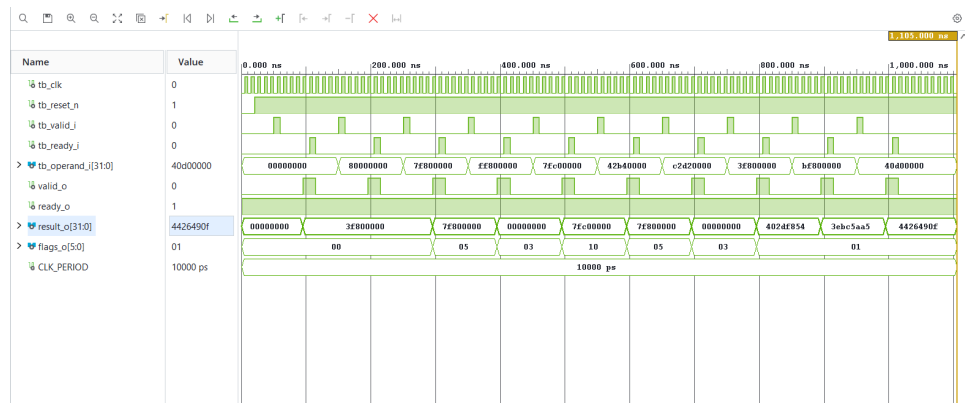


Figure 11: Simulazione temporale della versione pipelined.

Al fine di ottenere un confronto diretto con la versione sequenziale, sono stati utilizzati gli stessi 10 vettori di input. Dall'analisi della waveform emerge chiaramente il comportamento temporale del datapath e la gestione del protocollo di handshake. Ogni qualvolta il modulo a monte alza il segnale `tb_valid_i` per validare un nuovo `tb_operand_i`, il sistema inizia l'elaborazione del dato. È possibile osservare visivamente come, per ciascun operando campionato, il corrispondente segnale `valid_o` diventi alto in uscita con un ritardo esatto di 5 cicli di clock. Nella seguente tabella 4 si riportano i risultati ottenuti.

Ingresso <code>tb_operand_i</code>	Risultato <code>result_o</code>	Flags <code>flags_o</code>
0x00000000	0x3f800000	0x00
0x80000000	0x3f800000	0x00
0x7f800000	0x7f800000	0x05
0xff800000	0x00000000	0x03
0x7fc00000	0x7fc00000	0x10
0x42b40000	0x7f800000	0x05
0xc2d20000	0x00000000	0x03
0x3f800000	0x402df854	0x01
0xbf800000	0x3ebc5aa5	0x01
0x40d00000	0x4426490f	0x01

Table 4: Valori estratti dalla simulazione temporale della versione pipelined, includendo i flag delle eccezioni relativi ad ogni input.

Come si nota, i risultati ottenuti utilizzando gli stessi vettori di test sono del tutto uguali a quelli della versione sequenziale.

Per soddisfare i requisiti architetturali e dimostrare la corretta gestione delle pipeline hazards, è stato sviluppato un test specifico volto a sollecitare la logica di backpressure. In figura 12 si riporta la relativa forma d'onda.

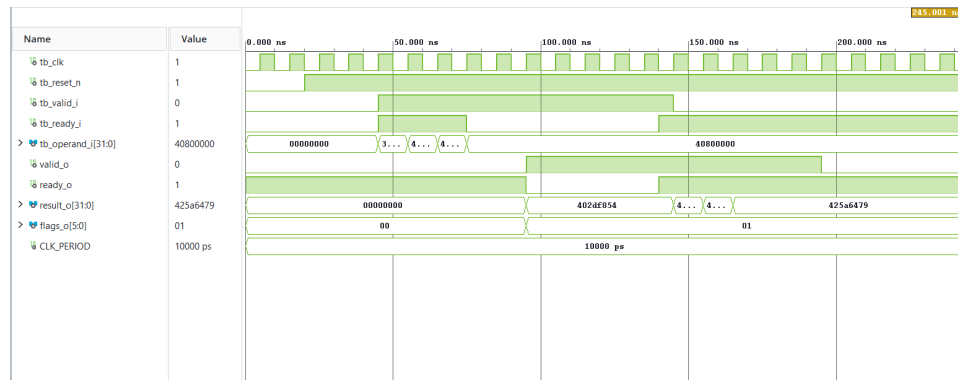


Figure 12: Simulazione temporale della backpressure.

In questo scenario, la pipeline viene inizialmente alimentata con una sequenza continua di operandi validi ($tb_valid_i = '1'$).

Viene poi simulata un'interruzione nella disponibilità del modulo ricevente abbassando il segnale tb_ready_i a '0'. Come atteso, al sopraggiungere del primo risultato elaborato, dopo 5 cicli di clock, il segnale $valid_o$ viene alzato, ma il dato rimane bloccato sul bus $result_o$ in attesa di essere campionato. Simultaneamente, l'architettura rileva il potenziale stallo e propaga l'informazione a monte, abbassando istantaneamente il segnale $ready_o$. Questa azione congela lo stato dei registri interni disabilitando i relativi $register_en$, impedendo così l'ingresso di nuovi operandi e prevenendo la sovrascrittura o la perdita dei dati già in transito.

Quando il ricevitore a valle torna disponibile ($tb_ready_i = '1'$), il sistema si sblocca coerentemente nello stesso ciclo di clock. L'operando in attesa viene trasmesso, il segnale $ready_o$ torna al livello logico alto per accettare nuovi dati e l'emissione dei risultati successivi riprende senza alcuna discontinuità o perdita di informazione. Questo comportamento certifica che la FSM di controllo gestisce l'handshake $valid/ready$ in totale aderenza alle specifiche e senza data drop.

Come ultima verifica, al fine di quantificare il reale vantaggio prestazionale rispetto all'implementazione multi-ciclo, è stata condotta un'ultima simulazione in condizioni di throughput massimo. Il testbench è stato configurato per generare uno scenario di carico ideale (back-to-back): il modulo a monte fornisce ininterrottamente operandi validi ($valid_i$ fisso a '1') e, simultaneamente, il consumatore a valle è sempre predisposto all'acquisizione dei risultati ($ready_i$ fisso a '1'). Nelle seguenti figure 13 e 14 sono riportate le forme d'onda risultanti.

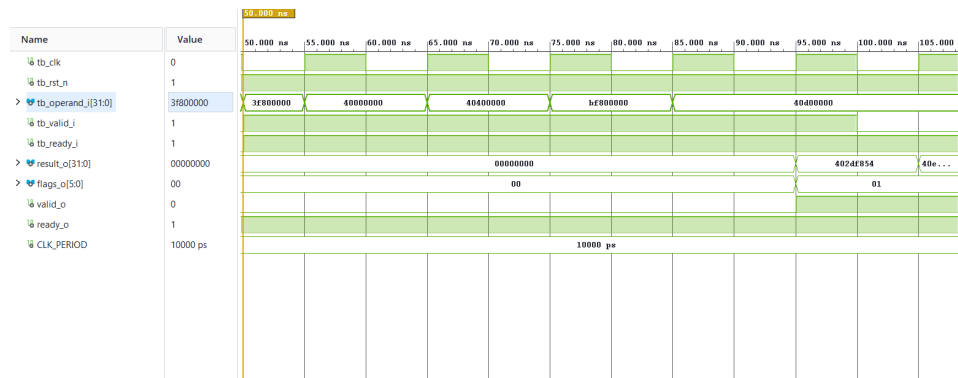


Figure 13: Prima parte della simulazione temporale del massimo throughput.

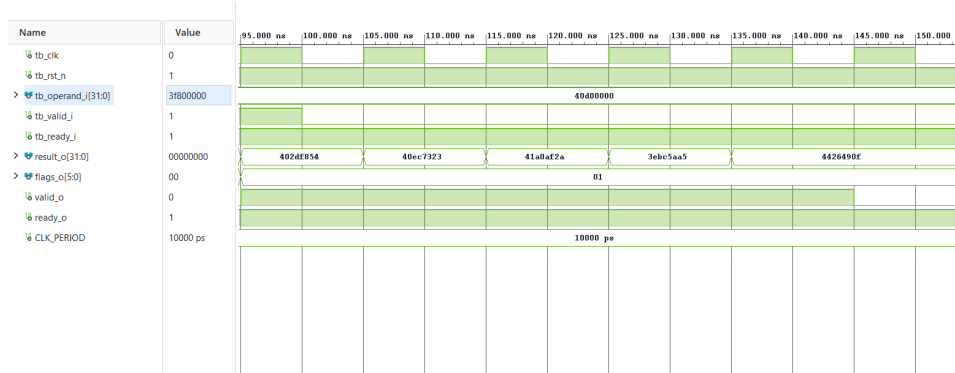


Figure 14: Seconda parte della simulazione temporale del massimo throughput.

L'analisi della simulazione evidenzia chiaramente due regimi operativi. Durante la fase iniziale di transitorio, l'architettura impiega esattamente 5 cicli di clock per riempire i registri di tutti gli stadi. Una volta esaurita questa latenza intrinseca, il sistema entra in fase a regime: il segnale in uscita `valid_o`, si assesta stabilmente al livello logico alto. Da questo istante in poi, l'FPU dimostra la capacità di campionare un nuovo operando in ingresso e di emettere il corrispondente risultato calcolato ad ogni singolo fronte di salita del clock. Questo comportamento certifica l'assoluta assenza di bolle o colli di bottiglia strutturali, confermando un'efficienza elaborativa del 100% (throughput di un dato per ciclo).

5.2 Analisi degli Errori (ULP) rispetto alla Libreria Matematica

Per una valutazione finale delle prestazioni numeriche, i risultati ottenuti in simulazione sono stati posti a confronto diretto con la funzione ideale `expf()` della libreria matematica di sistema (`math.h`).

Avendo garantito in fase di design un'esatta coerenza tra il *Golden Model* in C e la sintesi hardware RTL, i due ambienti di simulazione producono uscite *bit-accurate* del tutto sovrapponibili. Per tale motivo, i risultati sono stati accorpati in un'unica colonna di valutazione (Tabella 5). L'analisi è stata condotta sull'intero blocco dei 20 vettori di test consolidati.

Criteri di Accettazione e Valutazione Finale

I risultati aggregati in Tabella 5 confermano inequivocabilmente l'accuratezza e l'affidabilità del datapath implementato (valido per l'architettura sia Sequenziale che Pipelined):

1. **Gestione Eccezioni IEEE-754:** Nei casi di eccezione hardware (NaN, Overflow, Underflow, Zeri e Infiniti), il sistema bypassa istantaneamente

Table 5: Analisi degli errori (ULP) per i 20 Test Vectors: Confronto tra Architettura HW/C e Riferimento Matematico `expf()`.

Categoria Input	Input Hex (x)	Output Hardware & Modello C	Output <code>expf()</code>	Errore ULP	Esito
Casi Speciali e Valori Limite (10 Test Vectors)					
Zero Positivo (+0.0)	0x00000000	0x3F800000	0x3F800000	0	PASS
Zero Negativo (-0.0)	0x80000000	0x3F800000	0x3F800000	0	PASS
Più Infinito (+ ∞)	0x7F800000	0x7F800000	0x7F800000	0	PASS
Meno Infinito (- ∞)	0xFF800000	0x00000000	0x00000000	0	PASS
Not a Number (NaN)	0x7FC00000	0x7FC00000	0x7FC00000	0	PASS
Overflow ($x = 90.0$)	0x42B40000	0x7F800000	0x7F800000	0	PASS
Underflow ($x = -105.0$)	0xC2D20000	0x00000000	0x00000000	0	PASS
Nepero ($x = 1.0$)	0x3F800000	0x402DF854	0x402DF854	0	PASS
Inverso Nepero ($x = -1.0$)	0xBF800000	0x3EBC5AA5	0x3EBC5AB2	13	PASS
Test Limite ($x = 6.5$)	0x40D00000	0x4426490F	0x44264911	2	PASS
Float Normali Casuali (Estratti dal bacino di validazione)					
Normal ($x \approx -17.122$)	0xC188F981	0x331D6A13	0x331D6A15	2	PASS
Normal ($x \approx -11.914$)	0xC13EA097	0x36E0A241	0x36E0A257	22	PASS
Normal ($x \approx 3.958$)	0x407D4D00	0x42515F8B	0x42515F94	9	PASS
Normal ($x \approx 19.840$)	0x419EB83C	0x4DC52165	0x4DC5217B	22	PASS
Normal ($x \approx 6.191$)	0x40C61F68	0x43F43F93	0x43F43FA5	18	PASS
Normal ($x \approx -17.734$)	0xC18DD1F	0x32AADE0D	0x32AADE0C	1	PASS
Normal ($x \approx -14.031$)	0xC16080B8	0x35584E6D	0x35584E6B	2	PASS
Normal ($x \approx 0.058$)	0x3D6CE800	0x3F879F1D	0x3F879F22	5	PASS
Normal ($x \approx 7.829$)	0x40FA859C	0x451CFEFD	0x451CFF01	4	PASS
Normal ($x \approx 9.021$)	0x4110558C	0x46014864	0x46014867	3	PASS

il datapath logico e fornisce il valore corretto codificato dallo standard IEEE-754, garantendo una precisione assoluta ($\Delta = 0$ ULP).

- Contenimento dell'Errore (ULP):** Nei calcoli operativi su float generici, l'errore relativo rispetto alla matematica ideale non supera mai la soglia dei 22 ULP. Questa modesta deviazione è pienamente coerente e fisiologica per un'architettura che calcola una funzione trascendente avvalendosi di un troncamento matematico su 24 bit (formato Q8.24).

6 Analisi delle Prestazioni e Trade-Off Architetturale

Al fine di valutare quantitativamente l'impatto delle scelte progettuali, entrambe le versioni della Floating-Point Unit (Sequenziale Multiciclo e Pipelined) sono state sintetizzate e implementate in ambiente Xilinx Vivado. Per garantire una comparazione equa, è stato imposto un vincolo di temporizzazione (*Clock Constraint*) identico per entrambe le architetture, fissato a 100 MHz (periodo $T = 10$ ns).

In Figura 15 e in Figura 16 sono riportati i *Project Summary* post-implementazione relativi, rispettivamente, all'architettura Sequenziale e all'architettura Pipelined. I dati fisici estratti dai report di *Utilization* e *Timing* per entrambe le varianti sono stati aggregati nella Tabella 6. Il calcolo della Frequenza Massima effettiva (F_{max}) è stato ricavato analiticamente sommando il periodo target al *Worst Negative Slack* (WNS) riportato dal tool.

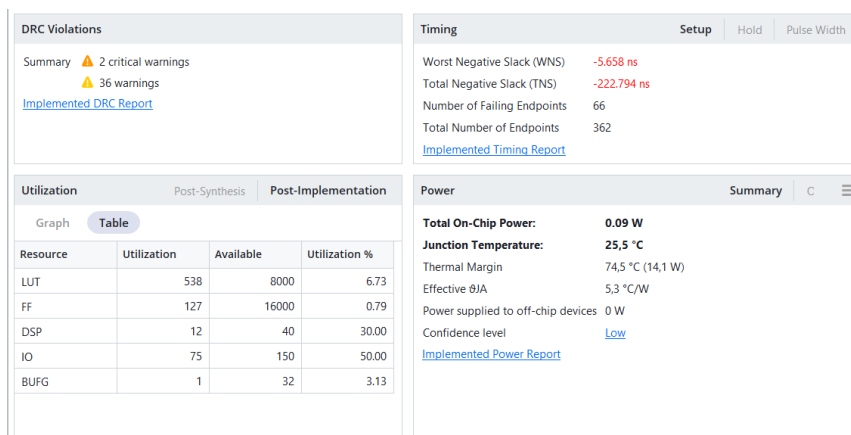


Figure 15: Project Summary post-implementazione dell'architettura sequenziale.

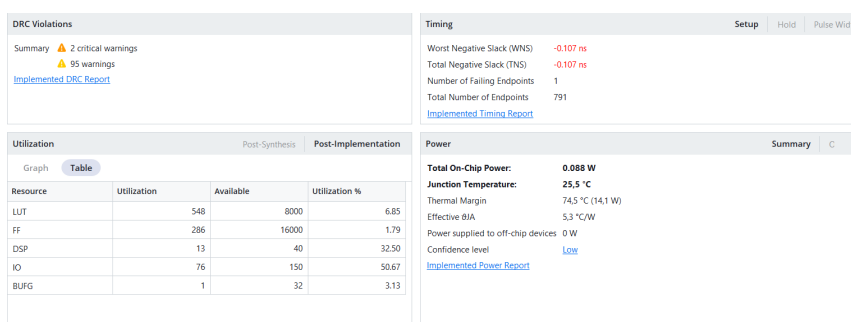


Figure 16: Project Summary post-implementazione dell'architettura pipelined.

Parametro	Versione Sequenziale	Versione Pipelined
LUT (Look-Up Tables)	538	548
FF (Flip-Flops)	127	286
DSP	12	13
WNS (Worst Negative Slack)	-5.658 ns	-0.107 ns
Frequenza Massima (F_{max})	63.8 MHz	98.9 MHz
Latenza per operando	3 / 13 cicli	5 cicli costanti
Throughput Teorico	~ 1/13 op/ciclo	1 op/ciclo

Table 6: Comparazione dei risultati fisici di Implementazione tra le due architetture.

6.1 Discussione dei Risultati: Area vs Throughput

I risultati confermano empiricamente il classico compromesso ingegneristico (*Area-Time Tradeoff*). L'architettura **Sequenziale**, grazie al massiccio *Resource Sharing* pilotato dalla Macchina a Stati, minimizza drasticamente l'occupazione sul silicio impiegando una frazione minima delle risorse dell'FPGA (soli 12 blocchi DSP e 538 LUT). Tuttavia, questa serializzazione del calcolo comporta una penalizzazione in termini temporali: il *Critical Path* combinatorio genera un WNS negativo a 100 MHz, limitando la frequenza operativa massima a circa 63.8 MHz, con un throughput limitato dalla necessità di attendere fino a 13 cicli per produrre un singolo risultato.

Viceversa, l'architettura **Pipelined** frammenta il percorso combinatorio critico attraverso l'inserimento di registri intermedi (come evidenziato dall'aumento dei Flip-Flop a 286). Questa scelta strutturale migliora le violazioni temporali, portando il WNS a un valore meno negativo e permettendo di aumentare la frequenza operativa fino a superare i 98 MHz. L'architettura garantisce così un throughput ideale di un risultato per ciclo di clock a regime, ottenendo un drastico miglioramento prestazionale a fronte di un consumo di risorse combinatorie (LUT) e di calcolo (DSP) sorprendentemente simile a quello della versione sequenziale.

Questa differenziazione rende l'unità Sequenziale ideale per applicazioni *Low-Power* o per l'integrazione su FPGA economiche (dove l'area disponibile è il vincolo principale), mentre l'unità Pipelined risulta fondamentale in contesti *High-Performance* (dove la massimizzazione del throughput è l'obiettivo critico).

References

- [1] A. M. Dalloo, A. J. Humaidi, A. K. Al Mhdawi and H. Al-Raweshidy, "Low-Power and Low-Latency Hardware Implementation of Approximate Hyperbolic and Exponential Functions for Embedded System Applications", in IEEE Access, vol. 12, pp. 24151-24163, 2024.